# ASUR

*The honest, firewalled AI Content Creation OS — one idea in, a finished video out, and it learns from every video.*

**COMPLETE GUIDE — VERSION 2 (October 2026)**

**What ASUR is, in one breath.** You type a one-line idea. ASUR researches it, figures out who it is for, picks a strategy, writes and scores 20+ hooks, writes the script, plans every shot, generates the visuals and voice, edits them on a timeline with captions and music, renders a platform-ready MP4, judges it honestly on 12 dimensions, packages it for YouTube or Instagram, and then **stops and waits for you**. After you publish, you feed back the real numbers and ASUR remembers which hooks worked, so the next video starts smarter.

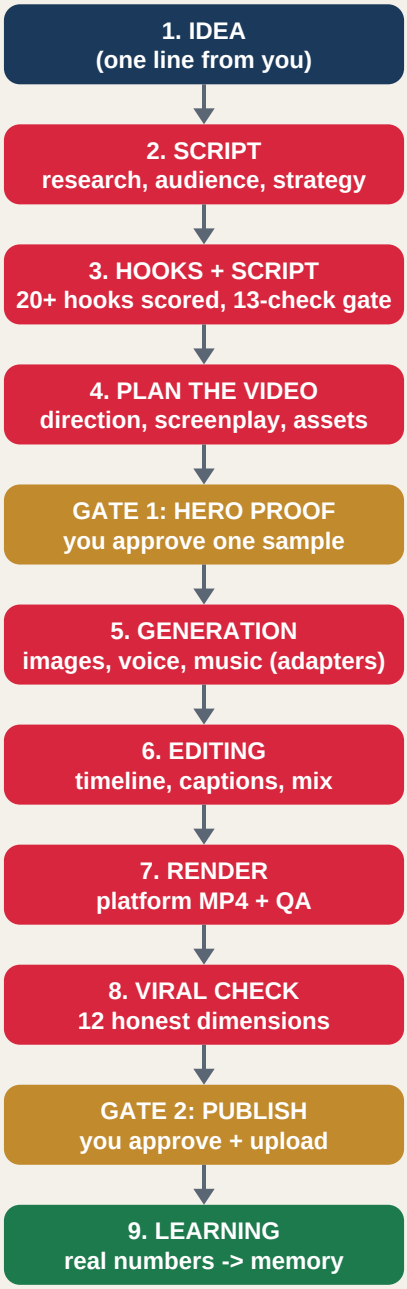
**Where it runs.** 100% on your own Mac. No login, no API keys, no cloud. The core is pure Python standard library; the heavy media tools (FFmpeg, SDXL, Kokoro voice, faster-whisper) are optional add-ons.

## What changed since version 1 of this guide

| Area | What is new |
|---|---|
| Folders | The project was reorganized. Side tools moved to **studio/**, PDFs to **guides/**, old output videos to **outputs/**, personal notes to **archive/**. Nothing was deleted. |
| YouTube folder | Now really exists: **asur/youtube/demo/** was produced by a real `asur render` run (final video, thumbnail, title, description, tags, metadata, content report). |
| New stages | P09 platform packs (Instagram Reels, YouTube Shorts, YouTube Long) and P10 real render + publish package are now in the guide. |
| IDEA 4 concepts | All 10 concepts (reconciliation every 4 hours, webhooks, gateway API, WebSocket, PPP, pooling, OOP abstract, WebRTC, SFU, signalling server) are mapped onto the ASUR flow with an honest status for each. |
| Tests | 309 automated tests pass (was 221 in version 1). |
| Old guide | Kept untouched as **guides/ASUR-Complete-Guide-v1.pdf**. |

# 1. The complete flow (remember this page)

Every ASUR video walks the same road. Red boxes are machine work, gold boxes are **you**, green is the learning loop that makes the next video better.




The arrow from LEARNING goes back to SCRIPT: the hook memory is read by the next idea.

# 2. The 26 stages and the code that runs each one

Every stage writes one **artifact** (a checksummed JSON file) into the project's append-only store `.script/artifacts/<kind>/vNNNN-*.json`. Nothing is ever overwritten; a change makes v0002.

| Stage | What happens | Code (inside asur/asur/) | Artifact |
|---|---|---|---|
| 1 | Idea | script/idea.py | idea |
| 2 | Research | script/research.py | research |
| 3 | Audience | script/audience.py | audience |
| 4 | Strategy | script/strategy.py | strategy |
| 5 | Hooks (20+ scored) | script/hooks.py, hook_memory.py | hooks |
| 6 | Script | script/script_builder.py | script |
| 7 | Script quality gate (13 checks) | script/quality_gate.py | quality report |
| 8 | Creative direction | generation/creative_direction.py | creative_direction |
| 9-10 | Timing + fact check | script/evidence.py | evidence classes |
| 11 | Visual screenplay | generation/visual_screenplay.py | visual_screenplay |
| 12-13 | Asset strategy + generation plan | asset_strategy.py, generation_plan.py | asset_plan, generation_plan |
| 14 | License guard | generation/license_guard.py | license block |
| 15-17 | Hero proof (one sample) | generation/hero_proof.py | hero_proof |
| 18 | **GATE 1 — you approve the hero proof** | core/gate.py | approval |
| 19 | Full generation (first paid-in-time step) | generation/dispatch.py + adapters/ | media + provenance |
| 20 | Section QA | generation/section_qa.py | section report |
| 21 | Edit + final render | editing/*, generation/render.py | edit_plan, final_render |
| 22 | Final QA (5 families) | generation/final_qa.py | final_qa |
| 23 | Viral check | viral_check/evaluate.py | viral_check |
| 24 | Publish package | learning/publish_package.py | publish_package |
| 25 | **GATE 2 — you approve publishing** | core/gate.py | approval |
| 26 | Performance + learning | performance.py, metrics_ingest.py, learning.py | performance, learning |

# 3. The state machine: 18 states, 2 human gates

The orchestrator (`orchestrator/states.py`) moves a project through 18 states. It only ever sees **control tokens** (state, gate, verdict, digest) — never the content itself. If anything is missing, it stops at HOLD or BLOCK (fail closed).

| States | Cost tier | Meaning |
|---|---|---|
| DRAFT -> INPUTS_VERIFIED -> RESEARCH_COMPLETE -> STRATEGY_APPROVED -> HOOK_APPROVED -> SCRIPT_APPROVED -> TIMING_VERIFIED | Cheap (stages 0-6) | Pure thinking. Free, instant, runs anywhere. |
| CREATIVE_DIRECTION_APPROVED -> SCREENPLAY_APPROVED -> HERO_PROOF_READY -> **HERO_PROOF_APPROVED (Gate 1)** | Moderate (7-18) | One sample is made so you can say yes before anything expensive. |
| FULL_GENERATION -> QA -> VIRAL_CHECK_APPROVED -> **PUBLISH_READY (Gate 2)** -> PUBLISHED -> PERFORMANCE_CAPTURED -> LEARNED | Expensive (19+) | GPU/time cost. Only after Gate 1. Publishing only after Gate 2. |

## The two human gates

**Gate 1 — Hero proof (stage 18).** ASUR makes one representative shot. You approve it before full generation spends time and compute.

**Gate 2 — Publish (stage 25).** ASUR builds the full upload package and stops. It never uploads by itself. You review the package in the youtube/ folder and publish.

Approvals are recorded with your local identity (`whoami` + git name/email). If you approve your own work alone, it is honestly recorded as `HOLD:SELF_APPROVED`.

## The firewall (why the judge can be trusted)

GENERATION and VIRAL CHECK never talk to each other. SCRIPT hands each one a filtered view (`tools/projection.py`): GENERATION_VIEW has the brief and quality floors but not the judge's criteria; VIRAL_VIEW has the finished video hashes and facts but every target is stripped by name. A hand-off queue with a hash chain (`tools/queue.py`) proves nothing was tampered with. So the maker cannot "study for the test".

# 4. Everything ASUR can do today

Honest status. **HAVE** = works now and is tested. **PARTIAL** = works with limits. **PLANNED** = designed, not built yet.

| Capability | Status |
|---|---|
| Turn one idea into a full plan: research, audience, strategy, 20+ scored hooks, script | HAVE |
| Write in English, Hindi, Hinglish, Indian-English, Marathi | HAVE |
| Script quality gate with 13 checks and evidence classes | HAVE |
| Creative direction, shot-by-shot visual screenplay, asset + generation plan | HAVE |
| License guard (blocks known license traps) + provenance on every asset | HAVE |
| Timeline editing with 15 primitives, captions inside safe area, audio mix | HAVE |
| Edit controls: accept, reject, modify, regenerate, lock, unlock, revert, branch, compare | HAVE |
| Render a real platform MP4 with FFmpeg (9:16 or 16:9) | HAVE |
| Burned-in captions | PARTIAL — needs an FFmpeg build with drawtext; otherwise renders clean cards and says so in provenance |
| Final QA in 5 families: visual, audio, text, content, platform | HAVE |
| Honest viral check: 12 dimensions, risk flags, verdict PASS / PASS-WITH-CHANGES / REJECT, never a fake % score | HAVE |
| Platform packs: Instagram Reels, YouTube Shorts, YouTube Long | HAVE |
| Publish package in youtube/<project>/package/ (video, thumbnail, title, description, tags, metadata, report) | HAVE |
| Learning: read your real numbers from a file, update hook memory | HAVE |
| Daily self-check (status, stdlib purity, firewall, lint, types, tests, drift) | HAVE |
| AI image generation (SDXL) | PARTIAL — adapter code present, needs `asur[generation]` install + model weights |
| AI voice (Kokoro), AI music (Stable Audio), speech-to-text (faster-whisper) | PARTIAL — adapter code being written right now (uncommitted work) |
| AI video clips (Wan 2.2) | PLANNED — stub only |
| Automatic uploading to YouTube / Instagram | NOT ALLOWED by design — you publish |

# 5. How to use it: the commands

Run everything from the **asur/** folder. ASUR is installed in editable mode, so `asur` and `python3 -m asur.cli` are the same thing.

## A. Write a script from an idea

```
asur script --idea "why most people waste money on AI tools" \
  --project . --project-id myvideo --language hinglish --platform youtube_shorts
```

## B. Run the orchestrator on a project

```
asur run --project . --project-id myvideo
```

## C. Make the youtube/ folder for real (demo, 2 commands)

```
python3 studio/seed_demo_project.py --project-id demo
asur render --project . --project-id demo --platform youtube_shorts
```

Step 1 builds every artifact from idea to edit plan plus the viral check. Step 2 renders the MP4 and writes the package, then stops at Gate 2. Platforms: instagram_reels, youtube_shorts, youtube_long.

## D. Reserved commands (not built yet)

```
asur generate # reserved: full generation from the CLI
asur viral-check # reserved: standalone viral check
```

## E. Health checks

```
python3 -m pytest -q # 309 tests
ruff check asur studio # lint
python3 -m asur.tools.daily_driver # daily self-check report
```

# 6. The youtube/ folder

**Why you could not see it before:** the folder is created by `asur render`, and that command had only ever run inside automatic tests (in temporary folders that are wiped). It had never been run on a real project in your asur folder. It has now been run, so the folder exists:

```
asur/youtube/
  demo/ <- one folder per project
    render/ <- rendered video versions
    _work/ <- temporary frames/cards
    package/ <- READY TO UPLOAD
      v0001-final-video.mp4
      v0001-thumbnail.jpg
      v0001-title.txt
      v0001-description.txt
      v0001-tags.txt
      v0001-platform-metadata.json
      v0001-content-report.json
```

**Versioning.** Files are never overwritten. Render again and you get v0002-*. **Git.** youtube/ is in .gitignore (videos are large and personal), so it lives on your disk only. **Every new project** gets its own folder: `youtube/<project-id>/`.

**Known limit on this Mac:** the Homebrew FFmpeg here has no *drawtext* filter, so captions are not burned in; the video renders with clean cards and the provenance says exactly that. Installing an FFmpeg built with libfreetype turns captions on.

# 7. The organized folder map

The project now has one clear rule: **the product** lives in `asur/`, **everything around it** has its own labelled shelf. Moves only — nothing was deleted.

| Folder | What lives there |
|---|---|
| asur/ | THE PRODUCT. The Python package (all stages, gates, firewall). |
| tests/ | 309 automated tests (+ integration tests that need FFmpeg). |
| docs/ | Design docs, frozen SPEC, ADRs (decisions), build plans, progress P00-P08. |
| studio/ | Hands-on creator tools: render_demo.py, edit_video.py, seed_demo_project.py, lyric_video/ (ingest, timing, sync, Remotion stub, your song project). |
| guides/ | These PDFs and the scripts that generate them (v1 kept). |
| youtube/ | Upload-ready packages per project (made by asur render). Not in git. |
| outputs/ | Old output videos: edit_out/ (6 edit styles) and render_out/ (demo reel). |
| archive/ | Kept but out of the way: SENIORITY_NOTES.md, the two stray 1-byte files user.email and user.name. |
| scripts/ | com.asur.daily.plist — macOS schedule for the daily self-check. |
| .script/ | The artifact store (every stage's JSON, versioned). Not in git. |
| .github/ | CI: tests, lint, and a stdlib-only import probe on every push. |
| root files | README, AGENTS (the constitution), CHANGELOG, DEFERRED (honest list of what is not real yet), SECURITY, CONTRIBUTING, LICENSE (MIT), pyproject.toml, MAP.md. |

## Inside the package: asur/asur/

| Module | Contents |
|---|---|
| core/ | canonical JSON + SHA-256, artifact envelope, gates, local identity, versioned workspace, media workspace (youtube/) |
| script/ | idea, research, audience, strategy, hooks, hook memory, script, quality gate, evidence |
| generation/ | creative direction, screenplay, asset + generation plans, hero proof, license guard, dispatch (router), adapters/, section + final QA, render |
| editing/ | timeline, 15 primitives, captions, audio mix, edit controls, OTIO adapter |
| viral_check/ | 12 dimensions, risk flags, verdict |
| learning/ | publish package, performance, metrics ingest, learning memory |
| platforms/ | base PlatformPack + Instagram Reels, YouTube Shorts, YouTube Long |
| orchestrator/ | 18 states, control-token orchestrator, run log |
| tools/ | firewall projections, hash-chained queue, status, daily driver |
| cli.py | the asur command: script, run, render (+ reserved generate, viral-check) |

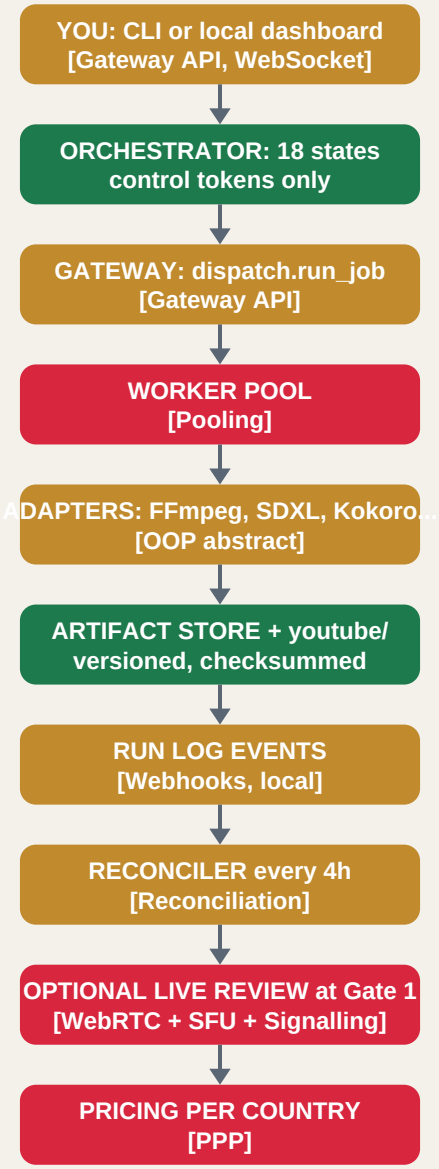
# 8. The 10 IDEA-4 concepts, mapped onto ASUR

Each concept below is placed where it belongs in the flow. The status is honest. One rule shapes everything: **ASUR-LOCAL-01 — 100% local, no network**. Concepts that need the internet can only exist as an optional, human-approved add-on, or **loopback-only** (127.0.0.1, your own machine).

| Concept | Simple meaning | Where it is in ASUR now | Status / next step |
|---|---|---|---|
| **OOP abstract** | One base contract, many implementations. | platforms/base.py PlatformPack (3 packs share it); generation adapters share one run contract via dispatch.py. | PARTIAL — next: formal abstract Adapter and Platform base classes. |
| **Gateway API** | One front door that routes every request. | generation/dispatch.py run_job: model name -> adapter. cli.py is the human front door. | PARTIAL — next: loopback-only HTTP gateway (stdlib). |
| **Pooling** | Reuse expensive things instead of recreating them. | Not yet. Sections render one after another; models reload each time. | PLANNED — worker pool for section renders; keep models loaded. |
| **Reconciliation every 4 hours** | Regularly compare "what should be" with "what is" and fix drift. | tools/daily_driver.py detect_drift + checksums in workspace + hash-chain in queue; scheduled by scripts/com.asur.daily.plist (once a day, weekdays 09:00). | PARTIAL — next: 4-hour interval (StartInterval 14400) and reconcile index vs files vs youtube/ packages. |
| **Webhooks** | "Call me when X happens." | Local equivalent exists: every step is written to the run log (orchestrator/observability.py). | PLANNED — local event hooks only. Real internet webhooks (e.g. YouTube callbacks) break LOCAL-01. |
| **WebSocket** | Live two-way stream. | Not yet. | PLANNED — loopback live progress feed from the run log to a local dashboard. |
| **Purchasing power parity** | Fair price per country. | Not code. Fits the business side (pricing tiers) and the 5 languages already supported. | PLANNED — pricing table; no network needed. |
| **WebRTC** | Live audio/video between browsers. | Not yet. | PLANNED, optional — live remote review at Gate 1. Needs network + human approval. |
| **SFU** | Server that forwards one stream to many viewers. | Not yet. | PLANNED, optional — several reviewers watch the hero proof together. |
| **Signalling server** | Introduces two WebRTC peers to each other. | Not yet. | PLANNED, optional — needed only if WebRTC is added; LAN-only first. |

# 9. How the concepts fit together (target design)

Read this top to bottom. Each layer uses the concept named in brackets. Solid today = **green**; partly there = **gold**; future = **red**.



# 10. The 8 laws and the honest judge

| Law | Meaning |
|---|---|
| LOCAL-01 | 100% local, key-free. Identity = whoami + git name/email. |
| HONEST-01 | No fake "% viral" number. Ever. |
| FIREWALL-01 | Maker and judge never talk directly. |
| GATE-01 | Fail closed. Missing evidence = HOLD or BLOCK, never PASS. |
| HUMAN-01 | Humans own inputs, approvals, history. No machine publishing. |
| PROV-01 | Every asset carries origin, model, prompt, license. |
| EXPLAIN-01 | Every score and rejection has a written reason. |
| VERSION-01 | Nothing silently overwritten. Append-only, checksummed. |

## The 12 viral-check dimensions

| | | | |
|---|---|---|---|
| Attention | Retention | Satisfaction | Value |
| Shareability | Saveability | Replayability | Originality |
| Authenticity | Visual Quality | Audience Fit | Platform Fit |

Verdict: any hard risk -> REJECT; any soft risk or weak dimension -> PASS-WITH-CHANGES; otherwise PASS. Each dimension is scored on input quality against documented signals, with its evidence class.

# 11. The studio: hands-on tools

| Tool | What it does | How to run |
|---|---|---|
| studio/seed_demo_project.py | Builds a full project (idea -> edit plan + viral check) so asur render can make the youtube/ package. | python3 studio/seed_demo_project.py --idea "..." --project-id demo |
| studio/render_demo.py | Idea -> ASUR script -> 9:16 reel with macOS voice, Ken-Burns motion and captions. | python3 studio/render_demo.py --idea "..." (writes outputs/render_out/) |
| studio/edit_video.py | Your clip -> 4 edit styles: punchy, cinematic, meme, beatcut (1080x1920), optional auto captions. | python3 studio/edit_video.py --input clip.mp4 (writes outputs/edit_out/) |
| studio/lyric_video/ingest.py | Song -> song.wav + draft lyrics. Rights gate (--i-have-rights) and lyrics gate. | python3 studio/lyric_video/ingest.py ... |
| studio/lyric_video/timing.py | Vocal isolation (Demucs) + forced alignment -> word timestamps, phrases, confidence report. --freeze locks it. | python3 studio/lyric_video/timing.py --project ... |
| studio/lyric_video/sync.py | Sync-test video with red word highlight to check timing by eye. | python3 studio/lyric_video/sync.py --project ... |

Full lyric-video walkthrough: guides/ASUR-Lyric-Video-Guide.pdf.

# 12. Honest status and what comes next

| Area | State |
|---|---|
| Phases P00-P10 | Built: foundations, SCRIPT, state machine + CLI, firewall + queue, generation core + hero proof, editing, full render + QA, viral check, learning + publish, platform packs, real render + package. |
| Tests | 309 passing. Lint clean on asur/ and studio/. |
| In progress (uncommitted) | P11 real prompts for generation: new adapter code for Kokoro, Stable Audio, faster-whisper and a test file (tests/test_generation_prompt_p11.py) that currently fails on purpose (tests written first). |
| Known issues | 2 type-check warnings (generation_plan.py, cli.py); FFmpeg on this Mac lacks drawtext so captions are not burned in. |
| Fixed in this update | Render no longer fails when FFmpeg lacks drawtext: it renders clean cards and records why (honest provenance). |

## Roadmap

| When | Goal | Status |
|---|---|---|
| Next | Finish P11 adapters (voice, music, captions) and make the P11 tests pass | IN PROGRESS |
| Next | Concepts that keep ASUR local: abstract base classes, worker pool, 4-hour reconciler, local event hooks, loopback gateway + WebSocket progress | PLANNED |
| Later | Wan 2.2 video clips; asur generate and asur viral-check commands | PLANNED |
| Optional | Live remote review (WebRTC + SFU + signalling) and PPP pricing — only behind a human gate, because they need the network | PLANNED |

This guide is generated from a script. Whenever ASUR changes, re-run `python3 guides/make_brand_guide.py` to produce an updated copy. It does not update on its own. The previous edition is kept as guides/ASUR-Complete-Guide-v1.pdf (generator: guides/make_brand_guide_v1.py).